

SPRINT 13 · AUTONOMOUS AGENT SYSTEMS · SESIÓN 1

Del while al grafo

LangGraph como orquestador: ReAct, checkpoints y la pausa humana. Empezamos la Live Review multiagente de calidad del aire.

HOY, EN 120 MINUTOS

Autonomía, pero con freno de mano

0–10	Apertura	Mapa S11 → S12 → S13
10–40	Pinceladas 1 y 2	ReAct en LangGraph · dónde va cada pieza del S12
40–55	Pincelada 3	Checkpoints, thread_id y MemorySaver
55–65	Descanso	
65–80	Pincelada 4	HITL: interrupt_before, update_state e invoke(None)
80–100	Live Review · Fase 0	La práctica, el entorno y el recorrido del grafo
100–120	Fase 1a y cierre	Escribimos _fallback_siguiente

DÓNDE ESTAMOS

Tres sprints, una misma pregunta que crece

11	¿Qué es un agente y cómo mantiene estado?	Bucle, estado, done, max_turns
12	¿Cómo actúa con herramientas?	Allowlist, max_steps, traza · LangGraph básico
13	¿Cómo orquesto con autonomía controlada?	Grafo, checkpoints, HITL y supervisor

Pregunta del sprint: **¿cómo doy más autonomía al agente sin perder el control de cuándo y qué ejecuta?**

El bucle de tools, dibujado



Dos detalles que se olvidan siempre

```
class Estado(TypedDict):  
    messages: Annotated[list, add_messages]  
    pasos: int  
  
def router(estados):  
    ultimo = estados["messages"][-1]  
    if getattr(ultimo, "tool_calls", None):  
        return "tools"  
    return END
```

El reducer acumula en lugar de pisar: cada nodo devuelve solo lo que cambia.

El tope es un contador en el estado. Si no hay tope, un prompt desafortunado da vueltas hasta agotar la cuota.

Nada se tira; cambia de sitio

run_tool_loop (el while)	Nodos agent y tools, o el mini-bucle dentro de un especialista
if function_calls	Edge condicional (router)
max_steps	Sigue, dentro del nodo o como contador del estado
TOOL_REGISTRY y ejecutar_tool	Se reutilizan tal cual: la allowlist no desaparece
procesar_turno	Mismo contrato; por dentro, invoke con thread_id
CLI y Streamlit	Siguen siendo clientes

OJO CON ESTO EN LA PRÁCTICA

El grafo de hoy no es un ReAct.

En la Live Review el StateGraph es el supervisor y sus especialistas.
El bucle modelo ↔ tools sigue existiendo, pero vive dentro de
cada especialista.

Checkpoints, thread_id y MemorySaver

Checkpoint	Foto del estado después de cada paso del grafo
thread_id	El identificador de la conversación: dos ids, dos hilos
MemorySaver	Guarda esas fotos en RAM; al cerrar el proceso se pierden
get_state(cfg)	Devuelve .values (estado actual) y .next (nodos pendientes)

Sin checkpointer no hay dónde pararse, y **sin pausa no hay human in the loop**.

Reducer y checkpointer no son lo mismo

Reducer

Acumula **dentro** de una corrida del grafo: los mensajes y el log de este invoke.

Checkpointer

Guarda el estado **entre** un invoke y el siguiente, por thread_id.

En Streamlit tendremos tres memorias y un hilo: messages pinta el chat, agent_state dirige la tarea, la traza registra, y el thread_id es lo que permite reanudar el grafo donde se quedó.

Pausar antes de actuar



La pausa

El grafo no entra en el nodo sensible. `get_state().next` incluye su nombre.

La decisión

`update_state` escribe `plan_aprobado` en el checkpoint.

La reanudación

`invoke(None)` sigue desde la pausa, sin mensaje nuevo.

Así se reanuda un grafo pausado

```
app.update_state(cfg, {"plan_aprobado": True}) # o False
resultado = app.invoke(None, cfg) # entra en aplicar

# ¿estamos en pausa?
snap = app.get_state(config_hilo(thread_id))
pendiente = "aplicar" in (snap.next or ())
```

Esto es lo que hay detrás de los botones **Aprobar** y **Rechazar** de la app.

LA LIVE REVIEW

Multiagente de calidad del aire con HITL

Carpeta: Practica_live_review/01_agente_langgraph_calidad_aire

Lo que viene dado

Las cuatro tools, los nodos del grafo, el router, el estado, la CLI, la app de Streamlit y el verificador.

Lo que implementas

`_fallback_siguiete` y `build_graph` en `graph.py`, y `procesar_turno` en `agent.py`

Cuatro fases

Entorno · grafo · orquestación · demostrar en CLI y Streamlit.

Arrancamos con el check en rojo

```
cd Sprint_13/Practica_live_review/  
cd 01_agente_langgraph_calidad_aire  
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt  
cp .env.example .env # GEMINI_API_KEY  
python main.py --check
```

Tres FAIL, uno por TODO:

```
[FAIL] _fallback_siguiete  
[FAIL] build_graph  
[FAIL] procesar_turno
```

Novedad en requirements: langgraph y langchain-google-genai.

FASE 0 · EL GRAFO QUE VAMOS A CABLEAR

Cinco nodos y una pausa



_fallback_siguiente: cuando el LLM no contesta bien

```
def _fallback_siguiente(estado):  
    if not (estado.get("notas_diagnostico") or "").strip():  
        return "diagnostico"  
    if not (estado.get("notas_practico") or "").strip():  
        return "practico"  
    if not (estado.get("respuesta") or "").strip():  
        return "sintetizar"  
    return "FINISH"
```

El supervisor pregunta al modelo qué rol toca. Si responde cualquier cosa, decide Python.

Es la regla de oro de siempre, ahora dentro de un grafo.

El verificador prueba los cuatro casos.

CIERRE DE LA SESIÓN 1

Tres ideas y un reto

1

Un grafo es estado, nodos y edges condicionales: el while que ya sabías escribir.

2

Sin checkpointer no hay pausa, y sin pausa no hay HITL.

3

El modelo propone el siguiente paso; Python decide si vale y cuándo se para.

Reto: intentad build_graph. Las pistas están plegadas al final del README; probad antes de abrirlas.

SPRINT 13 · AUTONOMOUS AGENT SYSTEMS · SESIÓN 2

Multiagente en marcha

Cableamos el grafo, escribimos el turno, aprobamos y rechazamos planes, y cerramos con multimodal y Streamlit.

HOY, EN 120 MINUTOS

Terminar el agente y demostrarlo

0–10	Repaso relámpago	Cinco preguntas en frío
10–22	Pincelada 5	Supervisor y especialistas
22–45	Fase 1b	build_graph en directo
45–60	Fase 2	procesar_turno en directo
60–70	Descanso	
70–100	Fase 3	Demo, pausa HITL, aprobar y rechazar, multimodal
100–120	Fase 4 y cierre	Streamlit, criterio de cierre y Project Break 2

REPASO RELÁMPAGO

Contestad en el chat, sin mirar apuntes

- 1 ¿Qué guarda un checkpoint y a qué va asociado?
- 2 ¿Qué hace `interrupt_before`?
- 3 ¿Cómo se reanuda un grafo pausado?
- 4 ¿Quién decide el siguiente nodo si el LLM contesta una palabra rara?
- 5 ¿Dónde vive ahora el bucle modelo ↔ tools?

Un nodo, un oficio

Estado compartido

El canal: pedido, notas de cada rol, plan, quién sigue, iteración y log.

Supervisor

Mira el estado y escribe **siguiente**. No hace el trabajo del especialista.

Vuelta al supervisor

Cada especialista tiene un edge fijo de regreso. Eso es el bucle del multiagente.

Y una salida explícita: **FINISH** más un tope de iteraciones que aplica Python si el modelo no cierra nunca.

build_graph: nodos, edges y pausa

```
grafo = StateGraph(EstadoGrafo)
grafo.add_node("supervisor", nodo_supervisor)
grafo.add_node("diagnostico", nodo_diagnostico) # y practico, sintetizar, aplicar

grafo.add_edge(START, "supervisor")
grafo.add_conditional_edges("supervisor", _router_supervisor,
    {"diagnostico": "diagnostico", "practico": "practico",
     "sintetizar": "sintetizar", "aplicar": "aplicar"})

grafo.add_edge("diagnostico", "supervisor") # y practico, sintetizar
grafo.add_edge("aplicar", END)

return grafo.compile(checkpointer=MemorySaver(), interrupt_before=["aplicar"])
```

El router ya viene dado

```
def _router_supervisor(estados):
    sig = (estados.get("siguiente") or "").strip()
    if sig == "FINISH":
        if not (estados.get("plan") or estados.get("respuesta")):
            return "sintetizar"
        return "aplicar"
    if sig in ("diagnostico", "practico", "sintetizar"):
        return sig
    return "sintetizar"
```

FINISH no va a END: va a **aplicar**, que es donde está la pausa.

Y si el supervisor dice FINISH sin plan, lo manda a sintetizar. Nunca se cierra en vacío.

procesar_turno: un mensaje, un invoke

```
app = get_app()
cfg = config_hilo(thread_id)
n = len(estado.get("traza") or []) + 1
try:
    inicial = estado_inicial_grafo(mensaje, max_steps=max_steps)
    resultado = app.invoke(inicial, cfg)
    nuevo = estado_desde_grafo(resultado, estado)
    nuevo = _marcar_hitl(nuevo, thread_id)
    nuevo.setdefault("traza", []).append(
        {"turno": n, "mensaje": mensaje, "status": "ok",
         "done": nuevo.get("done"), "pendiente_hitl": nuevo.get("pendiente_hitl"),
         "log": list(resultado.get("log") or []),
         "tools": list(resultado.get("traza_tools") or [])})
except Exception as e: # error + respuesta amigable + traza status error
```

Mismo contrato que en el Sprint 11; por debajo, un grafo con hilo.

FASE 3 · TRES COMANDOS

Lo que tiene que salir en la terminal

python main.py --check	Los tres TODO en [OK]
python main.py	Log supervisor → especialistas → sintetizar; plan; HITL auto-aprobado
python main.py --sin-auto-aprobar	pendiente_hitl: true y done: false
python main.py --interactivo	Escribes aprobar o rechazar a mano

Lee el **log** del resumen: cuenta la historia del turno iteración a iteración.

Cuatro iteraciones del supervisor

```
iter 1: supervisor → diagnostico
diagnostico: notas listas
iter 2: supervisor → practico
practico: notas listas
iter 3: supervisor → sintetizar
sintetizar: plan listo (pendiente HITL)
iter 4: supervisor → FINISH
```

Entre especialista y especialista, el control siempre vuelve al supervisor. Ese ir y venir es el multiagente.

Aprobar y rechazar

Aprobar

done pasa a true y la respuesta termina con el cierre. La conversación se da por hecha.

Rechazar

done sigue en false, el plan se vacía y las notas se limpian: el agente pide qué cambiar.

Lo que no cambia

El thread_id. Por eso el grafo sabe dónde estaba parado.

Comprueba la pausa con **--sin-auto-aprobar**: el proceso termina con el plan escrito y sin aplicar.

Una imagen como entrada de una tool

La tool

describir_imagen manda la imagen a Gemini y devuelve 2 o 3 frases de texto. El grafo no se entera de que es una imagen.

Hace falta un JPG

La tool lee **assets/imagen_demo.jpg**, y esa carpeta viene vacía: pon tú la imagen antes de la demo.

Cómo se dispara

Un mensaje que hable de imagen, foto o cartel. En la traza debe salir la tool.

Tres memorias y un hilo

st.session_state.messages	Las burbujas del chat
st.session_state.agent_state	Plan, log, traza, done y los flags de HITL
st.session_state.thread_id	El hilo del checkpointer: lo que permite reanudar

Con el plan pendiente, el chat se bloquea y los botones **Aprobar** y **Rechazar** aparecen en la barra lateral. «Nueva conversación» genera un thread_id nuevo.

CRITERIO DE CIERRE

Si lo explicas en voz alta, el sprint está cerrado

- 1 Qué hace el supervisor y qué no hace.
- 2 Por qué cada especialista vuelve al supervisor.
- 3 Qué guarda MemorySaver y a qué va asociado.
- 4 Qué ocurre exactamente con `interrupt_before` y cómo se reanuda.
- 5 Dónde viven ahora la `allowlist` y `max_steps`.
- 6 Qué cambia en el estado al rechazar un plan.
- 7 Por qué `done` lo sigue poniendo el código.

QUÉ PROBAR ANTES DE ENTREGAR

La tabla del README, resumida

--check	Los tres TODO en [OK]
Demo	Log del supervisor, tools en la traza y plan con dos viñetas o más
--sin-auto-aprobar	pendiente_hitl sin done
--interactivo + aprobar	Cierra con done y mensaje de cierre
--max-steps 1	Tools limitadas dentro de cada especialista
Sin red	Fuente: fallback en el preview de la API
Imagen demo	describir_imagen aparece en la traza

CIERRE DEL MÓDULO

Ahora, el Project Break 2.

Tenéis agente con estado, herramientas controladas, orquestación en grafo, memoria por hilo y una persona en el bucle. Eso es, de verdad, un sistema de agentes.

SPRINT 13 · MATERIAL ADICIONAL

Más tela que cortar

Los cinco notebooks en directo, seis experimentos sobre el grafo y la conversación de producción. Entre 60 y 90 minutos extra.

CÓMO REPARTIRLO

Bloques sueltos: coge los que te hagan falta

Los cinco notebooks en directo	45	Tras la teoría de cada bloque, o todos seguidos
Viaje en el tiempo con el historial	10	Después de hablar de checkpoints
Dos conversaciones a la vez	8	Justo después del historial
El tope del supervisor (MAX_ITER)	8	Tras ver el log de la demo
Romper el grafo a propósito	15	Lo mejor del material extra: déjalo si puedes
De la clase a producción	12	Cierre, antes del criterio
Retos de ampliación	10	Para terminar o como tarea

El bucle del Sprint 12, ya como grafo

```
# tres tools locales o con API pública
calcular · obtener_clima · buscar_dato

grafo.add_node("agent", nodo_agente)
grafo.add_node("tools", nodo_tools_contado)
grafo.add_conditional_edges("agent", router_react,
                           {"tools": "tools", END: END})
grafo.add_edge("tools", "agent")
```

Tres pruebas, de menos a más:

- 1 · raíz cuadrada de 1764 → una vuelta
- 2 · temperatura en Sevilla → tool con red
- 3 · dato + clima + potencia → varias vueltas

Mira siempre el contador de pasos y la lista de tools pedidas.

Ana y Bob, dos hilos que no se ven

Hilo ana-001

Se presenta en el turno 1 y en el turno 2 pregunta su nombre: el modelo lo sabe, porque el checkpoint trae el historial.

Hilo bob-001

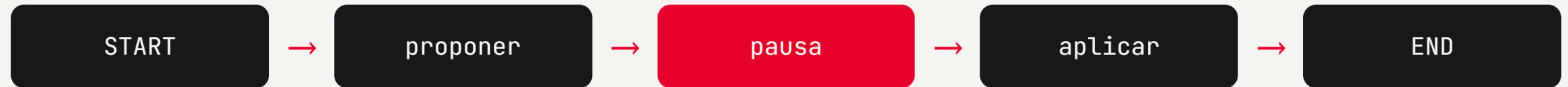
Pregunta su nombre sin presentarse. No dice «Ana». Otro thread_id es otra conversación.

get_state

Lee el último checkpoint sin gastar una llamada al modelo. Ideal para depurar.

La receta del config no se inventa: `{"configurable": {"thread_id": "..."}}}` en cada invoke.

Tres nodos y una pausa, sin ruido alrededor



Aprobar

update_state con aprobado a true, invoke con None, y el nodo cierra el plan.

Rechazar

Otro hilo, mismo grafo, aprobado en false: el nodo entra igual y deja constancia del rechazo.

Por qué este notebook

Es el mismo mecanismo de la práctica, pero en treinta líneas. Si alguien se ha perdido, aquí lo ve limpio.

El mismo patrón de la práctica, en dominio cultural

```
log del notebook:  
[supervisor] → cultura  
[cultura] generando notas...  
[supervisor] → practico  
[practico] generando notas...  
[supervisor] → sintetizar  
[sintetizar] uniendo...  
[supervisor] → FINISH
```

Aquí FINISH sí va a END: no hay pausa humana.
Compara con la práctica, donde FINISH va a aplicar.

Segundo pedido con otro thread_id: el log vuelve a empezar desde cero.

Si reutilizas el hilo anterior, el supervisor ve el plan cerrado y va directo a FINISH.

El contrato multimodal, en dos líneas

```
contents=[types.Content(role="user", parts=[
    types.Part.from_bytes(data=path.read_bytes(), mime_type="image/jpeg"),
    types.Part.from_text(text="Describe el cartel en 2 frases..."),
]])
```

Dentro del nodo

El grafo no sabe que hay una imagen: el nodo devuelve texto al estado, como cualquier otro.

Hace falta un cartel

El notebook lee
assets/cartel_demo.jpg y la
práctica assets/imagen_demo.jpg.
Prepáralos antes.

Lo que mejor luce

Una foto con texto: el modelo
lee horas, lugares y nombres
del cartel.

El historial de checkpoints

```
snap = app.get_state(cfg)
print(snap.next) # ('aplicar',)
print(snap.values["iteracion"]) # 4

hist = list(app.get_state_history(cfg))
print(len(hist)) # 9 checkpoints
print([h.next for h in hist[:5]])
```

Un turno deja **nueve** fotos: una por paso del grafo.

Puedes recorrerlas hacia atrás y ver el estado en cada momento.

Esto es auditoría gratis: qué sabía el agente y cuándo.

EXPERIMENTO 2 · DOS CONVERSACIONES A LA VEZ

El aislamiento, en vuestra propia práctica

```
python main.py --interactivo # terminal 1 → thread aire-ab12cd34  
python main.py --interactivo # terminal 2 → thread aire-9f88e011  
  
# en Python, sobre un hilo que no existe:  
app.get_state(config_hilo("inventado")).next # ()
```

Dos terminales, dos planes pendientes, cero interferencia. Lo mismo que pasa en Streamlit con «Nueva conversación».

EXPERIMENTO 3 · EL TOPE DEL SUPERVISOR

MAX_ITER a 2 en config.py

```
iter 1: supervisor → diagnostico
diagnostico: notas listas
iter 2: supervisor → FINISH (MAX_ITER)
sintetizar: plan listo (pendiente HITL)
iter 3: supervisor → FINISH (MAX_ITER)
```

El supervisor corta en la segunda iteración, sin pasar por practico.

Y mira lo que hace el router: FINISH sin plan lo manda a **sintetizar**, no a aplicar.

El agente cierra a medias, pero cierra con algo que enseñar.

EXPERIMENTO 4 · QUITAR LA FLECHA DE VUELTA

El fallo que no da ningún error

```
# comenta esta línea en build_graph  
# grafo.add_edge("diagnostico", "supervisor")
```

Resultado: done false, plan vacío, sin pausa, y el log con dos líneas.

El grafo compila, corre y termina... después de diagnostico.

Un nodo sin salida es un final. Nadie avisa.

EXPERIMENTO 5 · QUITAR EL INTERRUPT

Sin pausa, aplica igual.

El grafo entra en aplicar con plan_aprobado en false, su valor por defecto, y ejecuta la rama del rechazo: borra el plan sin preguntar a nadie. La pausa no es un adorno de la interfaz.

Lo que cambiarías mañana

MemorySaver en RAM	Checkpoint en SQLite o Postgres: la pausa sobrevive al reinicio
invoke y esperar	stream para ir pintando lo que hace cada nodo
Un turno = varias llamadas	Medir coste y latencia por turno antes de prometer nada
Aprobar en la barra lateral	Aprobar desde Teams, correo o un backoffice, con quién aprobó y cuándo
Tope de iteraciones	Tope, timeout por nodo y presupuesto por conversación

¿MULTIAGENTE PARA TODO?

No. Y conviene saber decirlo

Un agente con tools

Lo más barato y rápido. Suficiente cuando la tarea es una sola conversación con herramientas.

ReAct en grafo

Mismo alcance, pero con el ciclo explícito, trazable y con tope. El paso natural.

Supervisor y especialistas

Cuando los roles necesitan prompts e información distintos. Cuesta varias llamadas por turno.

Pregunta de arquitectura, no de moda: **¿qué parte del problema justifica una llamada más al modelo?**

Para los que van sobrados

Añade un cuarto especialista, por ejemplo alertas	Nodo nuevo, opción en el supervisor, destino en el router
Mueve la pausa a otro nodo	interrupt_before con otro nombre: ¿qué se puede aprobar ahora?
Añade una tool propia	La lista de tools y el prompt del especialista que la necesite
Respuestas personalizadas en el HITL	Aprobar, rechazar o pedir cambios: tres ramas en aplicar
Cambia el checkpointer a SQLite	Una línea en compile; comprueba que la pausa sobrevive